

Detailed Guide: Polish → English Tiny Transformer

This document explains the entire codebase in detail. It assumes you already know the basics of how Transformer models work (attention, encoder-decoder architecture, embeddings, positional encoding), but explains every implementation choice and code snippet so you can read, modify, and debug the program confidently.

Table of Contents

1. [Big Picture](#)
 2. [Files and Responsibilities](#)
 3. [Configuration \(config.py\)](#)
 4. [The Model \(src/model.py\)](#)
 5. [The Dataset \(src/dataset.py\)](#)
 6. [Training \(src/train.py\)](#)
 7. [Inference \(src/inference.py\)](#)
 8. [How Data Flows Through Training](#)
 9. [How to Run and Debug](#)
 10. [Common Pitfalls](#)
-

Big Picture

We want to train a neural machine translation model that takes Polish text and outputs English text. We do this with a classic **encoder-decoder Transformer** trained from scratch.

The data comes from Hugging Face:

- Dataset: HuggingFaceFW/finetranslations
- Language subset: pol_Latn (Polish text in Latin script)
- Polish source: og_full_text
- English target: translated_text

The tokenizer comes from:

- Helsinki-NLP/opus-mt-pl-en

This tokenizer already knows how to split Polish and English text into subword tokens, which saves us from building our own vocabulary.

The model is intentionally small so it can run on limited hardware like an AMD RX 680M integrated GPU.

Files and Responsibilities

```
Spikent/
├── config.py           # Hyperparameters and constants
├── requirements.txt    # Python packages
├── README.md          # Quick reference
├── DETAILED_GUIDE.md  # This file
└── src/
    ├── model.py       # The neural network
    ├── dataset.py     # Loading and formatting data
    ├── train.py       # The training loop
    └── inference.py   # Translating new sentences
```

Each file has one clear job. Keeping them separate makes the project easier to understand and modify.

Configuration (config.py)

This file contains two data classes: one for the model architecture and one for training settings.

```
from dataclasses import dataclass
```

```
@dataclass
class ModelConfig:
    src_vocab_size: int = 40000
    tgt_vocab_size: int = 40000
    d_model: int = 512
    nhead: int = 8
    num_encoder_layers: int = 4
    num_decoder_layers: int = 4
    dim_feedforward: int = 1024
    dropout: float = 0.1
    max_seq_len: int = 128
```

A `@dataclass` is just a convenient way in Python to define a class that mostly holds values. It automatically creates `__init__`, `__repr__`, and comparison methods.

What each field means

Field	Meaning
src_vocab_size	Number of tokens the encoder can read. We overwrite this later with the tokenizer's real vocabulary size.
tgt_vocab_size	Number of tokens the decoder can output. Same as above, overwritten later.
d_model	Size of every vector inside the model. Every token becomes a 512-dimensional vector.
nhead	Number of attention heads in each multi-head attention layer.
num_encoder_layers	How many Transformer encoder blocks are stacked.
num_decoder_layers	How many Transformer decoder blocks are stacked.
dim_feedforward	Size of the hidden layer inside the feed-forward network of each Transformer block.
dropout	Probability of randomly zeroing neurons during training to prevent overfitting.
max_seq_len	Maximum number of tokens per sentence. Longer sentences are truncated.

The default values create a model with roughly 30-40 million parameters, which is small enough for consumer GPUs.

```
@dataclass
```

```
class TrainConfig:
```

```
    dataset_name: str = "HuggingFaceFW/finetranslations"
```

```
    dataset_subset: str = "pol_Latn"
```

```
    src_column: str = "og_full_text"
```

```
    tgt_column: str = "translated_text"
```

```
    lang_score_column: str = "og_language_score"
```

```
    max_samples: int | None = 2_000_000
```

```
    max_seq_len: int = 128
```

```
    tokenizer_name: str = "Helsinki-NLP/opus-mt-pl-en"
```

```
    batch_size: int = 64
```

```
    gradient_accumulation_steps: int = 2
```

```
    num_epochs: int = 3
```

```

learning_rate: float = 3e-4
weight_decay: float = 0.01
label_smoothing: float = 0.1
warmup_steps: int = 4000
max_grad_norm: float = 1.0

output_dir: str = "./checkpoints"
save_every_n_steps: int = 5000
eval_every_n_steps: int = 2500
log_every_n_steps: int = 100

seed: int = 42
num_workers: int = 4
use_amp: bool = True

```

What each field means

Field	Meaning
dataset_name, dataset_subset	Where to get the data from Hugging Face.
src_column	Column name for Polish text.
tgt_column	Column name for English translation.
lang_score_column	Column with a confidence score that the source is actually Polish.
max_samples	Stop after streaming this many examples. The full Polish subset has 56.8M rows, but 2M is plenty for a small model.
batch_size	Number of examples processed together on one GPU.
gradient_accumulation_steps	How many batches to process before updating weights. Effective batch size = $\text{batch_size} \times \text{gradient_accumulation_steps}$.
num_epochs	How many times to pass over the dataset.
learning_rate	Step size for the optimizer.
weight_decay	L2 regularization strength.
label_smoothing	Softens target labels from 1.0 to 0.9, improving generalization.
warmup_steps	

Field	Meaning
	Learning rate rises linearly for this many steps, then follows cosine decay.
max_grad_norm	Clips gradients so their norm does not exceed this value, stabilizing training.
output_dir	Where checkpoints are saved.
save_every_n_steps	Save a checkpoint every N steps.
eval_every_n_steps	Reserved for future validation runs.
log_every_n_steps	Print training statistics every N steps.
seed	Random seed for reproducibility.
num_workers	Number of background processes for data loading.
use_amp	Whether to use Automatic Mixed Precision (FP16) for faster training.

The Model (src/model.py)

This file defines the neural network. It is a clean, from-scratch Transformer built on top of PyTorch's `nn.Transformer`.

Positional encoding

Transformers have no built-in sense of word order. A sentence like “kot je mysz” and “mysz je kot” would look identical without positional information. Positional encoding adds a unique vector to each position so the model knows which word is where.

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int = 512, dropout:
        float = 0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) /
            d_model)
        )
        pe = torch.zeros(1, max_len, d_model)
```

```

        pe[0, :, 0::2] = torch.sin(position * div_term)
        pe[0, :, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.pe[:, : x.size(1)]
        return self.dropout(x)

```

This uses the standard sinusoidal positional encoding from the original “Attention Is All You Need” paper.

- position has shape [max_len, 1] and contains position indices 0, 1, 2, ...
- div_term is a decreasing sequence of frequencies.
- pe is precomputed once and registered as a buffer (not a parameter, so it is not updated by the optimizer).
- In forward, we slice pe to match the actual input length and add it to the embeddings.

The full model

```

class TransformerTranslator(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.cfg = cfg
        self.d_model = cfg.d_model
        self.max_seq_len = cfg.max_seq_len

        self.embedding = nn.Embedding(cfg.src_vocab_size,
                                       cfg.d_model)
        self.pos_enc = PositionalEncoding(cfg.d_model,
                                           cfg.max_seq_len, cfg.dropout)

        self.transformer = nn.Transformer(
            d_model=cfg.d_model,
            nhead=cfg.nhead,
            num_encoder_layers=cfg.num_encoder_layers,
            num_decoder_layers=cfg.num_decoder_layers,
            dim_feedforward=cfg.dim_feedforward,
            dropout=cfg.dropout,
            batch_first=True,
        )

        self.fc_out = nn.Linear(cfg.d_model, cfg.tgt_vocab_size)
        self._init_parameters()

```

Here is what each component does:

- nn.Embedding: A lookup table. Each integer token ID is converted into a dense vector of size d_model.
- PositionalEncoding: Adds position information.

- nn.Transformer: PyTorch's built-in Transformer. With `batch_first=True`, input tensors have shape (batch, seq_len, features) instead of (seq_len, batch, features).
- `fc_out`: A linear projection from `d_model` down to vocabulary size. The output is a score for every possible next token.
- `_init_parameters()`: Initializes weights with Xavier uniform initialization, which helps training stability.

The forward pass

```
def forward(
    self,
    src: torch.Tensor,
    tgt: torch.Tensor,
    src_mask: torch.Tensor | None = None,
    tgt_mask: torch.Tensor | None = None,
    src_key_padding_mask: torch.Tensor | None = None,
    tgt_key_padding_mask: torch.Tensor | None = None,
) -> torch.Tensor:
    src_emb = self.pos_enc(self.embedding(src) *
                           math.sqrt(self.d_model))
    tgt_emb = self.pos_enc(self.embedding(tgt) *
                           math.sqrt(self.d_model))

    if tgt_mask is None:
        tgt_mask =
            self.transformer.generate_square_subsequent_mask(
                tgt.size(1)
            ).to(tgt.device)

    out = self.transformer(
        src=src_emb,
        tgt=tgt_emb,
        tgt_mask=tgt_mask,
        src_key_padding_mask=src_key_padding_mask,
        tgt_key_padding_mask=tgt_key_padding_mask,
    )
    return self.fc_out(out)
```

Step by step:

1. `self.embedding(src)` converts Polish token IDs to vectors.
2. Multiplying by `sqrt(d_model)` matches the scaling used in the original Transformer paper.
3. `self.pos_enc(...)` adds positional information.
4. The same embedding table is reused for the English decoder input.
5. `generate_square_subsequent_mask` creates a causal mask so the decoder can only attend to previous positions, not future ones. This is essential for autoregressive generation.
6. `src_key_padding_mask` and `tgt_key_padding_mask` tell the model which positions are padding and should be ignored.

7. The final linear layer `fc_out` produces logits over the English vocabulary.

Inference: translating a sentence

```
@torch.inference_mode()
def translate(
    self,
    src: torch.Tensor,
    bos_id: int,
    eos_id: int,
    max_len: int | None = None,
    beam_size: int = 1,
) -> torch.Tensor:
    if max_len is None:
        max_len = self.max_seq_len

    self.eval()
    batch_size = src.size(0)
    device = src.device

    if beam_size == 1:
        tgt = torch.full((batch_size, 1), bos_id,
            dtype=torch.long, device=device)
        finished = torch.zeros(batch_size, dtype=torch.bool,
            device=device)

        for _ in range(max_len - 1):
            logits = self.forward(src, tgt)[: , -1, :]
            next_token = logits.argmax(dim=-1).unsqueeze(1)
            tgt = torch.cat([tgt, next_token], dim=1)
            finished |= next_token.squeeze(1) == eos_id
            if finished.all():
                break
        return tgt

    raise NotImplementedError("Beam search not implemented; use
        beam_size=1.")
```

This is **greedy decoding**, the simplest way to generate text:

1. Start with a batch of [BOS] tokens.
2. Run the model to get logits for the next token.
3. Pick the token with the highest score (argmax).
4. Append it to the output sequence.
5. Repeat until EOS is produced for every example or `max_len` is reached.

`@torch.inference_mode()` disables gradient computation, saving memory and speed during translation.

The Dataset (src/dataset.py)

This file handles loading data from the internet, filtering it, and turning it into PyTorch tensors.

Loading the dataset

```
def load_translation_dataset(
    dataset_name: str,
    subset: str,
    split: str = "train",
    streaming: bool = True,
):
    return load_dataset(
        dataset_name,
        name=subset,
        split=split,
        streaming=streaming,
        trust_remote_code=True,
    )
```

`load_dataset` comes from the `datasets` library. We use `streaming=True` because the full dataset is many terabytes. Streaming means we download examples one at a time instead of loading everything into RAM.

Tokenizing one example

```
def tokenize_example(
    example: dict,
    tokenizer: PreTrainedTokenizer,
    src_column: str,
    tgt_column: str,
    max_seq_len: int,
) -> dict:
    src = example[src_column]
    tgt = example[tgt_column]

    src_tokens = tokenizer(
        src,
        truncation=True,
        max_length=max_seq_len,
        padding=False,
    )

    with tokenizer.as_target_tokenizer():
        tgt_tokens = tokenizer(
            tgt,
            truncation=True,
            max_length=max_seq_len,
```

```

        padding=False,
    )

    return {
        "input_ids": src_tokens["input_ids"],
        "attention_mask": src_tokens["attention_mask"],
        "labels": tgt_tokens["input_ids"],
    }

```

- `tokenizer(...)` converts Polish text into a list of integer token IDs.
- with `tokenizer.as_target_tokenizer()`: is a Hugging Face convention. For some tokenizers, the target language may need special handling. In our case the source and target use the same tokenizer, but this keeps the code correct and idiomatic.
- `truncation=True` cuts long sentences down to `max_seq_len`.
- `padding=False` means we do not pad here; padding is done later when grouping examples into a batch.

Iterable dataset

```

class TranslationIterableDataset(IterableDataset):
    def __init__(
        self,
        hf_dataset,
        tokenizer: PreTrainedTokenizer,
        src_column: str,
        tgt_column: str,
        max_seq_len: int,
        max_samples: int | None = None,
        lang_score_column: str | None = None,
        min_lang_score: float = 0.9,
    ):
        ...

    def __iter__(self) -> Iterator[dict]:
        count = 0
        for example in self.dataset:
            if self.max_samples is not None and count >= self.max_samples:
                break

            if not example.get(self.src_column) or not example.get(self.tgt_column):
                continue

            if (
                self.lang_score_column
                and example.get(self.lang_score_column, 1.0) < self.min_lang_score
            ):

```

```

        continue

    tokenized = tokenize_example(...)

    if len(tokenized["input_ids"]) < 3 or
len(tokenized["labels"]) < 3:
        continue

    count += 1
    yield tokenized

```

IterableDataset is PyTorch's way of representing a stream of data. We iterate over the Hugging Face dataset, filter bad examples, tokenize, and yield dictionaries.

Filtering rules:

- Skip examples with missing source or target.
- Skip examples where the Polish language confidence score is below 0.9.
- Skip examples that are too short after tokenization (fewer than 3 tokens).

Collating batches

```

def collate_fn(batch: list[dict], pad_token_id: int) -> dict[str,
    torch.Tensor]:
    max_src = max(len(item["input_ids"]) for item in batch)
    max_tgt = max(len(item["labels"]) for item in batch)

    input_ids = []
    attention_mask = []
    labels = []

    for item in batch:
        src = item["input_ids"]
        tgt = item["labels"]
        src_pad_len = max_src - len(src)
        tgt_pad_len = max_tgt - len(tgt)

        input_ids.append(src + [pad_token_id] * src_pad_len)
        attention_mask.append([1] * len(src) + [0] * src_pad_len)
        labels.append(tgt + [-100] * tgt_pad_len)

    return {
        "input_ids": torch.tensor(input_ids, dtype=torch.long),
        "attention_mask": torch.tensor(attention_mask,
            dtype=torch.long),
        "labels": torch.tensor(labels, dtype=torch.long),
    }

```

Because sentences have different lengths, we pad them to the longest sentence in the batch.

- `input_ids`: source token IDs padded with `pad_token_id`.
- `attention_mask`: 1 for real tokens, 0 for padding.
- `labels`: target token IDs padded with -100, which is PyTorch's standard "ignore this position" value for loss functions.

Building the dataloader

```
def build_dataloader(
    dataset: IterableDataset,
    tokenizer: PreTrainedTokenizer,
    batch_size: int,
    num_workers: int = 0,
) -> DataLoader:
    return DataLoader(
        dataset,
        batch_size=batch_size,
        collate_fn=lambda batch: collate_fn(batch,
        tokenizer.pad_token_id),
        num_workers=num_workers,
        pin_memory=torch.cuda.is_available(),
    )
```

`Dataloader` groups examples into batches. The `collate_fn` is called automatically to pad each batch. `pin_memory` speeds up CPU-to-GPU transfers when CUDA is available.

Training (src/train.py)

This is the largest file. It wires together the model, data, optimizer, scheduler, and checkpointing.

Imports and helper functions

```
from accelerate import Accelerator
from torch import optim
from torch.optim.lr_scheduler import LambdaLR
from transformers import AutoTokenizer

from config import ModelConfig, TrainConfig
from src.dataset import TranslationIterableDataset,
    build_dataloader, load_translation_dataset
from src.model import TransformerTranslator
```

- `Accelerator` from Hugging Face handles mixed precision, multi-GPU, and gradient accumulation with minimal code changes.
- `AutoTokenizer` downloads and loads the Helsinki tokenizer.

```
def set_seed(seed: int):
    random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
```

Setting seeds makes experiments reproducible.

```
def count_parameters(model: nn.Module) -> int:
    return sum(p.numel() for p in model.parameters() if
                p.requires_grad)
```

This counts trainable parameters, which tells us how large the model is.

Learning rate scheduler

```
def get_linear_warmup_cosine_decay_scheduler(
    optimizer: optim.Optimizer,
    warmup_steps: int,
    total_steps: int,
):
    def lr_lambda(step: int) -> float:
        if step < warmup_steps:
            return float(step + 1) / float(max(1, warmup_steps))
        progress = float(step + 1 - warmup_steps) / float(max(1,
            total_steps - warmup_steps))
        return 0.5 * (1.0 + torch.cos(torch.tensor(progress *
            3.14159265))).item()

    return LambdaLR(optimizer, lr_lambda)
```

This scheduler does two things:

1. **Warmup:** For the first `warmup_steps`, the learning rate rises linearly from near zero to the target value. This prevents early training instability.
2. **Cosine decay:** After warmup, the learning rate follows a cosine curve down to near zero. This helps convergence.

Saving checkpoints

```
def save_checkpoint(
    accelerator: Accelerator,
    model: nn.Module,
    optimizer: optim.Optimizer,
    scheduler,
    step: int,
    output_dir: str,
    is_best: bool = False,
):
    save_dir = Path(output_dir) / f"step_{step}"
```

```

if accelerator.is_main_process:
    save_dir.mkdir(parents=True, exist_ok=True)

accelerator.wait_for_everyone()
unwrapped_model = accelerator.unwrap_model(model)
accelerator.save(
    {
        "step": step,
        "model_state_dict": unwrapped_model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "scheduler_state_dict": scheduler.state_dict(),
    },
    save_dir / "checkpoint.pt",
)
if accelerator.is_main_process:
    tokenizer.save_pretrained(save_dir)
    (save_dir / "DONE").touch()

```

Checkpoints save:

- Model weights
- Optimizer state (so training can resume)
- Scheduler state
- The tokenizer

`accelerator.unwrap_model(model)` gets the raw model from underneath Accelerate's wrapper so we can save its weights cleanly.

The training function

```

def train(cfg: TrainConfig, model_cfg: ModelConfig):
    set_seed(cfg.seed)
    accelerator = Accelerator(
        mixed_precision="fp16" if cfg.use_amp else "no",
        gradient_accumulation_steps=cfg.gradient_accumulation_steps,
    )

```

Accelerator is initialized with FP16 mixed precision and gradient accumulation. This means we can use larger effective batch sizes and train faster.

```

global tokenizer
tokenizer = AutoTokenizer.from_pretrained(cfg.tokenizer_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

```

We load the tokenizer. Some models do not define a padding token, so we fall back to the end-of-sequence token. Padding and EOS are treated the same during training because padding positions are masked out.

```

model_cfg.src_vocab_size = len(tokenizer)
model_cfg.tgt_vocab_size = len(tokenizer)
model_cfg.max_seq_len = cfg.max_seq_len

```

```

model = TransformerTranslator(model_cfg)

```

The tokenizer's vocabulary size overrides the placeholder values in ModelConfig.

```

optimizer = optim.AdamW(
    model.parameters(),
    lr=cfg.learning_rate,
    weight_decay=cfg.weight_decay,
    betas=(0.9, 0.98),
    eps=1e-9,
)

```

AdamW is the standard optimizer for Transformers. The betas and eps values are common defaults from the original Transformer paper and modern implementations.

```

dataset = load_translation_dataset(
    cfg.dataset_name,
    cfg.dataset_subset,
    split="train",
    streaming=True,
)

train_dataset = TranslationIterableDataset(
    dataset,
    tokenizer,
    cfg.src_column,
    cfg.tgt_column,
    cfg.max_seq_len,
    max_samples=cfg.max_samples,
    lang_score_column=cfg.lang_score_column,
)

train_loader = build_dataloader(
    train_dataset,
    tokenizer,
    cfg.batch_size,
    num_workers=cfg.num_workers,
)

```

The data pipeline is built: load the streaming dataset, wrap it in the iterable dataset, then create a dataloader.

```

effective_batch_size = cfg.batch_size *
    cfg.gradient_accumulation_steps
total_samples = cfg.max_samples or 1_000_000
total_steps = (total_samples // effective_batch_size) *
    cfg.num_epochs

```

We estimate how many optimizer steps will happen. This is needed by the scheduler.

```
criterion = nn.CrossEntropyLoss(
    ignore_index=-100,
    label_smoothing=cfg.label_smoothing,
)
```

The loss function compares predicted token probabilities against the true next token. -100 positions are ignored, and label smoothing prevents the model from becoming overconfident.

```
model, optimizer, train_loader, scheduler =
    accelerator.prepare(
        model, optimizer, train_loader, scheduler
    )
```

`accelerator.prepare` wraps everything for mixed precision, distributed training, and device placement.

The main training loop

```
for epoch in range(1, cfg.num_epochs + 1):
    model.train()
    for batch in train_loader:
        with accelerator.accumulate(model):
            src = batch["input_ids"]
            tgt = batch["labels"]
            src_mask = batch["attention_mask"].bool()

            bos_id = tokenizer.bos_token_id or
tokenizer.pad_token_id
            decoder_input = torch.full(
                (tgt.size(0), 1),
                bos_id,
                dtype=torch.long,
                device=tgt.device,
            )
            decoder_input = torch.cat([decoder_input,
tgt[:, :-1]], dim=1)

            logits = model(
                src=src,
                tgt=decoder_input,
                src_key_padding_mask=~src_mask,
                tgt_key_padding_mask=(decoder_input ==
tokenizer.pad_token_id),
            )

            loss = criterion(
                logits.reshape(-1, logits.size(-1)),
                tgt.reshape(-1),
```



```

parser.add_argument("--batch-size", type=int,
                    default=TrainConfig.batch_size)
parser.add_argument("--output-dir", type=str,
                    default=TrainConfig.output_dir)
args = parser.parse_args()

train_cfg = TrainConfig(
    max_samples=args.max_samples,
    num_epochs=args.epochs,
    batch_size=args.batch_size,
    output_dir=args.output_dir,
)
model_cfg = ModelConfig()
train(train_cfg, model_cfg)

```

The script accepts a few command-line overrides so you can quickly test with smaller values.

Inference (src/inference.py)

After training, this script loads a checkpoint and translates new Polish sentences.

```

def load_checkpoint(checkpoint_dir: str, device: str | None =
                    None):
    if device is None:
        device = "cuda" if torch.cuda.is_available() else "cpu"

    tokenizer = AutoTokenizer.from_pretrained(checkpoint_dir)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token

    cfg = ModelConfig(
        src_vocab_size=len(tokenizer),
        tgt_vocab_size=len(tokenizer),
    )
    model = TransformerTranslator(cfg)

    ckpt = torch.load(
        f"{checkpoint_dir}/checkpoint.pt",
        map_location=device,
        weights_only=True,
    )
    model.load_state_dict(ckpt["model_state_dict"])
    model.to(device)
    model.eval()
    return model, tokenizer, device

```

Important: the model must be reconstructed with the same architecture as during training. We load the tokenizer from the checkpoint directory to guarantee the same vocabulary.

```
def translate_text(
    model: TransformerTranslator,
    tokenizer: AutoTokenizer,
    text: str,
    device: str,
    max_len: int = 128,
) -> str:
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        max_length=max_len,
    )
    src = inputs["input_ids"].to(device)

    bos_id = tokenizer.bos_token_id or tokenizer.pad_token_id
    eos_id = tokenizer.eos_token_id or tokenizer.pad_token_id

    output = model.translate(src, bos_id=bos_id, eos_id=eos_id,
                             max_len=max_len)
    output = output[0].tolist()

    if eos_id in output:
        output = output[: output.index(eos_id)]
    return tokenizer.decode(output, skip_special_tokens=True)
```

- Tokenize the Polish input.
 - Move it to the correct device.
 - Call `model.translate` for greedy decoding.
 - Truncate the output at the first EOS token.
 - Decode token IDs back into readable English text.
-

How Data Flows Through Training

Here is the full pipeline for one training step:

1. **Hugging Face dataset** streams a raw example:

```
{
    "og_full_text": "Dzień dobry.",
    "translated_text": "Good morning.",
    "og_language_score": 0.98
}
```

2. **Dataset loader** filters and tokenizes it:

```
{
  "input_ids": [12, 345, 67],
  "attention_mask": [1, 1, 1],
  "labels": [89, 901, 234]
}
```

3. **DataLoader** groups examples into a batch and pads them.

4. **Training loop** builds decoder_input from labels:

- labels: [89, 901, 234, <pad>]
- decoder_input: [<BOS>, 89, 901, 234]

5. **Model forward pass**:

- Encoder processes Polish tokens.
- Decoder predicts English tokens one by one using teacher forcing.
- Output logits shape: (batch, seq_len, vocab_size).

6. **Loss computation** compares predictions against labels.

7. **Backpropagation** updates the model weights.

How to Run and Debug

Install dependencies

```
cd Spikent
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Dry-run with a tiny dataset

```
python -m src.train \
  --max-samples 1000 \
  --epochs 1 \
  --batch-size 8 \
  --output-dir ./test-checkpoints
```

This completes in minutes and lets you verify the pipeline works without downloading much data.

Real training run

```
python -m src.train \
  --max-samples 2000000 \
  --epochs 3 \
```

```
--batch-size 64 \  
--output-dir ./checkpoints
```

Translate after training

```
python -m src.inference \  
--checkpoint ./checkpoints/best \  
--text "Kocham programowanie."
```

Common Pitfalls

1. **CUDA out of memory:** Reduce `batch_size` or `max_seq_len` in `config.py`, or increase `gradient_accumulation_steps`.
 2. **Tokenizer pad token is None:** The code handles this by setting `pad_token = eos_token`, but some tokenizers may behave differently. If you see errors, inspect `tokenizer.pad_token_id` after loading.
 3. **weights_only=True fails on older PyTorch:** If inference crashes with a weights-only error, remove that argument from `torch.load`.
 4. **No validation set:** The current code trains without measuring validation BLEU. Add a validation stream if you want early stopping or hyperparameter tuning.
 5. **Streaming dataset has no length:** Iterable datasets do not report their length. That is why we estimate `total_steps` from `max_samples`.
 6. **Multi-GPU runs:** Accelerator handles this automatically, but `num_workers` in the `DataLoader` may need to be 0 in some streaming configurations.
-

Summary

This project builds a small Polish-to-English Transformer from scratch using PyTorch and Hugging Face tools. The code is split into focused files: configuration, model, data loading, training, and inference. Data streams from Hugging Face, is tokenized with an existing Marian tokenizer, padded into batches, and fed through an encoder-decoder Transformer with teacher forcing. The model is kept small so it can be trained cheaply in the cloud and run locally on modest hardware.